

M1.1: Singleton Pattern & volatile DCL

Theory: Guarantees a class has only one instance. Double-Checked Locking (DCL) reduces synchronization overhead by delaying instantiation.

Details: The instance variable must be declared 'volatile' to prevent CPU instruction reordering. Without it, the 3-step creation ('alloc', 'init', 'assign') can be reordered to 1->3->2, letting concurrent threads obtain a reference to a partially initialized object.

Trade-off: DCL requires complex dual-null checks and synchronizations. Alternatively, the static inner Holder pattern achieves thread-safe lazy loading via classloader initialization locks.

M1.3: Dynamic Proxy & CGLIB

Theory: Controls access to a target object by dynamically creating proxies at runtime, powering Java AOP frameworks.

Details: JDK dynamic proxy implements target interfaces using reflection ('java.lang.reflect.Proxy'). CGLIB overrides methods by generating a subclass of the target at runtime via the ASM bytecode editor.

Trade-off: CGLIB cannot proxy 'final' classes or 'final/private' methods. Reflection calls in JDK proxy add stack frames, introducing microsecond latency overheads in high-frequency loops.

M1.5: Facade & Flyweight Patterns

Theory: Facade simplifies client interfaces to complex subsystems; Flyweight minimizes heap memory by caching and sharing immutable intrinsic states.

Details: Facade orchestrates subsystem interfaces. Flyweight partitions states into intrinsic (shareable) and extrinsic (context-dependent) data, caching instances in registries (e.g., 'Integer.valueOf').

Trade-off: Flyweight introduces CPU lookup overhead in registries. If cached objects are short-lived, registry retention might block GC reclamation.

M2.3: Template Method & Command Patterns

Theory: Template Method defines algorithm steps in a superclass, deferring hook implementations to subclasses; Command encapsulates requests as standalone command objects.

Details: Template Method relies on 'final' orchestrations. Command stores receivers and actions, enabling queueing, logging, and undo operations.

Trade-off: Template method subclassing is rigid; changes to the superclass template impact all children. Command pattern can cause class bloat due to separate command objects.

M2.1: Observer Pattern

Theory: Establishes a one-to-many relationship where state changes in a Subject trigger notifications to registered Observers.

Details: Subject maintains a list of Observers, pushing state updates (Push Model) or notifying them to query updates (Pull Model).

Trade-off: Synchronous observer loops can be blocked by a single slow observer. Production systems prefer asynchronous pub-sub architectures backed by message queues.

M1.2: Factory Method & Abstract Factory

Theory: Factory Method delegates object instantiation to subclasses; Abstract Factory provides an interface to create families of related products without specifying concrete classes.

Details: Decouples client logic from concrete types. Supports extending product suites while adhering to Open-Closed principles.

Trade-off: Multiplies class counts, increasing system complexity. For simple, non-varying instantiations, using direct constructors ('new') is cleaner.

M1.4: Adapter & Decorator Patterns

Theory: Adapter bridges incompatible interfaces; Decorator dynamically attaches new responsibilities to an object via composition.

Details: Adapter translates data signatures. Decorator holds a reference to the wrapped object and implements the same interface, extending behavior at runtime (e.g., 'BufferedInputStream').

Trade-off: Heavy nesting of decorators increases debugging complexity and produces long stack traces during exception profiling.

M2.2: Strategy & State Patterns

Theory: Strategy encapsulates interchangeable algorithms; State allows an object to alter its behavior when its internal state changes.

Details: Strategy objects are injected into contexts. State objects hold a back-reference to context, switching context states automatically post-action.

Trade-off: Creates many small strategy/state classes. For systems with only 2-3 states, basic 'if-else' conditionals are easier to maintain.

M2.4: Chain of Responsibility Pattern

Theory: Avoids coupling request senders and receivers by passing requests along a chain of handlers until one processes it.

Details: Handlers contain references to downstream nodes. Implemented as named filters or interceptors executing pre/post actions (e.g., Servlet 'FilterChain').

Trade-off: Long chains increase execution stack depth. Unchecked recursions without exit boundaries can cause JVM 'StackOverflowError' crashes.

M2.5: Mediator & Memento Patterns

Theory: Mediator caps multi-object interaction complexity by routing communications through a central hub; Memento captures object states for rollback without exposing internals.

Details: Mediator acts as an communication hub. Memento comprises the Originator (creates state), Memento (read-only state), and Caretaker (stores history).

Trade-off: Memento consumes significant heap space if tracking state history of large objects. Mediator can easily swell into an unmaintainable "God class".

M3.1: DTO & Entity Decoupling

Theory: Decouples persistence layers from presentation. DTOs transfer data across networks without exposing domain Entity logic.

Details: Entities contain business rules. DTOs are flat data containers. MapStruct is used to compile mappings at build time to avoid reflection overheads.

Trade-off: Introduces conversion boilerplate. For simple CRUD microservices, bypass mapping by exposing entities directly to minimize complexity.

M3.4: Circuit Breaker & Fallback Patterns

Theory: Guards system availability by failing fast and executing fallbacks when downstream dependencies exceed error rate thresholds.

Details: States: Closed (traffic passes), Open (traffic blocked; returns fallback data), Half-Open (sends test requests to check recovery).

Trade-off: Failures are masked via fallbacks, which can degrade UX (e.g., serving static data instead of live updates). Thresholds must be tuned carefully.

M4.2: JMM & volatile Memory Barriers

Theory: The Java Memory Model defines how threads interact with main memory, guaranteeing visibility, ordering, and consistency.

Details: 'volatile' forces memory visibility. JVM inserts hardware-level memory barriers (e.g., 'StoreLoad') to flush CPU store buffers and invalidate L1/L2 caches.

Trade-off: 'volatile' does not guarantee atomicity for compound operations (e.g., 'i++'). Atomic variables or lock syncs must be used for mutators.

M3.2: DAO & Repository Patterns

Theory: Abstraction of data access. DAO handles raw SQL CRUD operations per table; Repository acts as an collection-like interface managing domain aggregates.

Details: Repository aggregates multiple DAOs, mapping tabular rows into domain entities to enforce aggregate boundaries.

Trade-off: Repository models add mapping latency. For bulk writes or heavy reporting, bypass the repository and use raw batch SQL.

M3.5: CQRS & Event Sourcing

Theory: Splits read and write paths. Event Sourcing stores every state change as an immutable event sequence instead of overwriting database rows.

Details: Write path appends events to an Event Store, asynchronously updating the read view. Read path queries optimized denormalized read stores.

Trade-off: Replaying millions of events to reconstruct aggregate state is slow. Snapshot patterns are required to store checkpoints every N events.

M4.3: ThreadPoolExecutor Configuration

Theory: Pool threads to reuse them, avoiding the overhead of creating and destroying threads under heavy loads.

Details: Parameters: 'corePoolSize', 'maximumPoolSize', and 'workQueue'. When queue saturates, the pool executes rejection handlers.

Trade-off: Using unbound queues (like default 'LinkedBlockingQueue') under massive load causes queue backup, consuming all heap space and causing OOM crashes.

M3.3: Intercepting Filter & Front Controller

Theory: Web request lifecycle architecture. Front Controller provides centralized routing; Intercepting Filters pre-process cross-cutting concerns (auth, logging).

Details: Spring MVC's 'DispatcherServlet' routes HTTP requests, utilizing HandlerInterceptors to execute preHandle/postHandle hooks.

Trade-off: The centralized Front Controller is a traffic bottleneck. Slow sync operations inside filters can starve the controller's request thread pool.

M4.1: Java 1:1 Thread Mapping

Theory: In HotSpot JVM, each Java thread maps directly to one OS kernel thread, utilizing OS-level schedulers.

Details: Creating Java threads incurs system call overhead. Each thread allocates a 1MB private stack for frame variables.

Trade-off: Spawning threads excessively causes memory pressure and high CPU context-switch latency. Java thread execution must be managed via thread pools.

M4.4: ThreadLocal Memory Leaks

Theory: Allocates private variables per thread, avoiding lock contention by isolating access within the thread.

Details: Thread references thread-local values via weak reference keys in 'ThreadLocalMap' entries.

Trade-off: Weak keys are GC-reclaimed, but strong value references persist within Thread object chains. Threads reused in pools will leak memory unless 'remove()' is called.

M5.1: ReentrantLock & AQS CLH Queue

Theory: Powered by AbstractQueuedSynchronizer (AQS), ReentrantLock manages state via volatile variables and a double-linked CLH wait queue.

Details: Threads execute CAS to acquire locks. Failed threads are queued as nodes, parked via 'LockSupport.park()', and unparked post-release.

Trade-off: Non-fair mode increases throughput via lock hijacking but can starve queued threads. Fair mode prevents starvation but adds context-switch overhead.

M5.4: Synchronized Lock Inflation

Theory: HotSpot JVM optimizes 内置 locks by adjusting locking mechanisms based on thread contention levels.

Details: Transitions from Biased (record Thread ID in Mark Word) to Lightweight (CAS thread stack lock record) to Heavyweight (OS mutex sleep).

Trade-off: Inflation is one-way. Under high contention, the overhead of bias checks and lightweight spin loops can be avoided by disabling bias checks via JVM flags.

M6.1: Reactive Streams Backpressure

Theory: Non-blocking asynchronous streams require backpressure to protect memory-limited subscribers from being overwhelmed by fast publishers.

Details: Subscriber controls flow rate by calling 'Subscription.request(n)' to request n items. Publisher only sends up to n items.

Trade-off: Backpressure requires full-chain support. Any sync blocking library (e.g., blocking JDBC drivers) breaks the pipeline, causing memory pressure.

M6.4: ForkJoinPool Work Stealing

Theory: High-performance scheduler executing divide-and-conquer parallel tasks.

Details: Each thread maintains a Deque. Workers fetch tasks from the head (LIFO) and steal tasks from the tail (FIFO) of other busy threads' Deques.

Trade-off: Work stealing reduces lock contention, but heavy recursion creates excessive short-lived task objects, increasing GC pressure under tight heap allocations.

M5.2: CAS & ABA Versioning

Theory: Optimistic locking using hardware-level Compare-And-Swap instructions to update variables without blocking threads.

Details: Checks if current value equals expected value before swapping. Retries via CPU spin loops.

Trade-off: Vulnerable to ABA transitions (value changes A -> B -> A, appearing unchanged). Solved by tracking versions using 'AtomicStampedReference'.

M6.2: Flux & Mono Thread Schedulers

Theory: Flux represents 0..N elements; Mono represents 0..1 element. Schedulers manage execution threads.

Details: 'subscribeOn' schedules source emission threads; 'publishOn' switches threads for downstream operators.

Trade-off: Excessive thread switching via 'publishOn' degrades performance due to context-switch latency. Group operators onto shared schedulers.

M5.3: ConcurrentHashMap Segment Locking

Theory: Thread-safe Map using fine-grained bucket synchronization to allow concurrent reads and writes.

Details: Maps keys to bins. Only the head node of a hash bucket is locked using 'synchronized', leaving other buckets free. Converts bins to Red-Black trees when size >8.

Trade-off: Database resize and rehashing operations are expensive, though ConcurrentHashMap mitigates this by distributing transfer tasks across write threads.

M5.5: Deadlock Detection & Prevention

Theory: Block state where threads are stuck waiting for locks held by each other, violating circular wait boundaries.

Details: JVM checks for deadlocks by looking for loops in the Wait-for Graph.

Trade-off: Resolving active deadlocks requires killing threads. Focus on prevention by enforcing lock ordering and using 'tryLock(timeout)' limits.

M6.3: CompletableFuture Pipelines

Theory: Asynchronous task chaining API supporting parallel and sequential execution branches.

Details: Executes tasks in the ForkJoin common pool. Supports task combination via 'thenCombine' and error capture via 'exceptionally'.

Trade-off: Sharing 'ForkJoinPool.commonPool()' across independent subsystems can lead to starvation if one blocking task consumes all threads. Dedicated pools are required.

Zone T1: Java Concurrency APIs

```
Proxy.newProxyInstance(), AbstractQueuedSynchronizer.compareAndSetState(), ThreadPoolExecutor.execute(), CompletableFuture.allOf(), ThreadLocal.remove(), ConcurrentHashMap.putVal(), ForkJoinPool.commonPool()
```

Zone T2: System Concurrency Faults

```
out_of_memory metaspace, concurrency deadlock, thread_pool_exhaustion, thread_local_leak, aba_problem, concurrent_modification, race_condition
```